Lab Report – EE-170L Computer Programming (Lab 02)

NAME	NAEEM UL HAQ
ID	0665
SEMESTER	2nd
LAB report	6
DATE	21/4/2026

1. Objectives

- Understand the concept and syntax of the **for loop** in C++.
- Learn how to implement **nested for loops** for structured and repeated operations.
- Apply loops to solve problems such as counting, pattern printing, and mathematical computations.

2. Theory

2.1 The for Loop

A for loop is a control structure that allows code to be executed repeatedly based on a condition. It is typically used when the number of iterations is known beforehand.

Syntax:

```
for (initialization; condition; update) {
    // statements to execute
}
```

- initialization: Executed once at the beginning. Used to initialize loop control variables.
- Condition: Evaluated before each iteration. If true, the loop body executes.
- Update: Executed after each iteration. Typically used to modify the loop variable.

Flow Diagram:

Code

```
Start
  |
Initialization
  |
Condition True? ---- No ---> Exit Loop
  |
  v
Statements
  |
Update
  |
Go back to Condition
```

2.2 Nested for Loops

A nested loop is a loop inside another loop. The inner loop completes all its iterations for each iteration of the outer loop. Useful for multi-dimensional data like tables and patterns.

3. Programs Implemented

3.1 Countdown from 5 to 1

```
#include <iostream>          // Include input-output stream
```

```
using namespace std;          // Use standard namespace


int main() {
   for (int i = 5; i > 0; i--) {  // Initialize i=5; loop while i>0; decrement i
 cout << i <<end l; // Print current value of i
   }
   cout << "outside loop now"; // Print after loop ends
   return 0;          // End of program
}
```

Output:

5

4

3

2

1

outside loop now

3.2 Prime Number Check

```
#include <iostream>
using namespace std;


int main() {
```

```
int num;                        // Variable to store user input
cout << "Enter a number please\n";     // Prompt user
cin >> num;                          // Read input
int flag = 0;                       // Flag to track if number is not prime

for (int i = 2; i < num; i++) {        // Loop from 2 to num-1
    if (num % i == 0) {              // Check if num is divisible by i
        flag = 1;                   // Set flag if divisible
        break;                      // Exit loop early
    }
}

if (flag == 1) {
cout << num << " is not a prime number\n";          // Not prime
} else {
    cout << num << " is a prime number\n";      // Prime
    }
    return 0;
}
```

3.3 Nested Loop Pattern Printing

```
#include <iostream>
```

```
using namespace std;

int main() {
    for (int num = 7; num > 0; num--) {        // Outer loop from 7 to 1
        for (int j = 1; j <= num; j++) {      // Inner loop from 1 to num
            cout << j << " ";                 // Print numbers in row
        }
        cout << "\n";                         // New line after each row
    }
    return 0;
}
```

Output:

```
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

4. Lab Task

4.1 While Loop: Display Numbers from 1 to 10

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;                  // Initialize counter
    while (i <= 10) {           // Loop while i <= 10
        cout << i << endl;      // Print current number
        i++;                    // Increment counter
    }
    return 0;
}
```

4.2 Multiplication Table using Nested For Loops

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {        // Outer loop for rows
```

```
    for (int j = 1; j <= 10; j++) {     // Inner loop for columns
        cout << i * j << "\t";          // Print product with tab
    }
    cout << endl;                       // New line after each row
}
return 0;
}
```

4.3 Sum of Numbers from 1 to N

4.4 Triangle Pattern Printing

```
#include <iostream>
using namespace std;

int main() {
    int n, sum = 0;                     // Declare variables
    cout << "Enter a positive integer: ";  // Prompt user
    cin >> n;                           // Read input

    for (int i = 1; i <= n; i++) {      // Loop from 1 to n
        sum += i;                       // Add i to sum
    }
```

```
    cout << "Sum = " << sum;            // Display result
    return 0;
}
```

5. Conclusion

- The lab provided hands-on experience with **for loops**, **while loops**, and **nested loops**.
- Students learned how to control program flow using loop constructs.
- Practical applications included counting, checking prime numbers, generating multiplication tables, and printing patterns.
- These concepts are foundational for solving repetitive tasks efficiently in programming.